

BANKING MANAGEMENT SYSTEM

PROJECT TITLE

Console-Based Banking Management System

OBJECTIVE

The objective of this project is to design and develop a Banking Management System that enables efficient management of customer accounts and banking transactions. The system provides functionalities such as account creation, customer management, deposits, withdrawals, fund transfers, and transaction history maintenance using Java, JDBC, and MySQL.

TECHNOLOGIES USED

- Java
- JDBC
- MySQL Database
- IntelliJ IDEA
- Object-Oriented Programming

PROJECT DESCRIPTION

The Banking Management System is a console-based application developed using Java and MySQL. The system allows users to perform various banking operations through a menu-driven interface. Customer and account information are stored securely in a MySQL database and accessed using JDBC connectivity.

The application supports customer registration, account creation, deposits, withdrawals, fund transfers, and transaction history tracking. Unique client IDs and account numbers are generated automatically to ensure proper account management. The project demonstrates the practical implementation of Java programming, database management, and Object-Oriented Programming concepts in a real-world banking scenario.

FEATURES

- Create Customer Account
- View Customer Details
- View Account Details
- Display All Customers
- Display All Accounts
- Deposit Money
- Withdraw Money
- Transfer Money
- Transaction History Tracking
- Random ID and Account Number Generation

DATABASE DESIGN

The Banking Management System uses MySQL as its backend database. The database is designed to store customer information, account details, savings account information, and transaction records.

DATABASE NAME: banking_system

Table 1: clients

Stores customer information.

Column Name	Data Type	Description
client_id	VARCHAR(20)	Unique Client ID (Primary Key)
name	VARCHAR(100)	Customer Name
address	VARCHAR(200)	Customer Address

Table 2: accounts

Stores account information.

Column Name	Data Type	Description
account_number	BIGINT	Account Number (Primary Key)
balance	DOUBLE	Current Account Balance
client_id	VARCHAR(20)	Customer ID (Foreign Key)
transaction_date	TIMESTAMP	Account Creation Date

Relationship:

- One Client can have one or more Accounts.
- client_id references clients(client_id).

Table 3: savings

Stores savings account information.

Column Name	Data Type	Description
account_number	BIGINT	Savings Account Number
interest_rate	DOUBLE	Interest Rate

Relationship:

- account_number references accounts(account_number).

Table 4: transactions

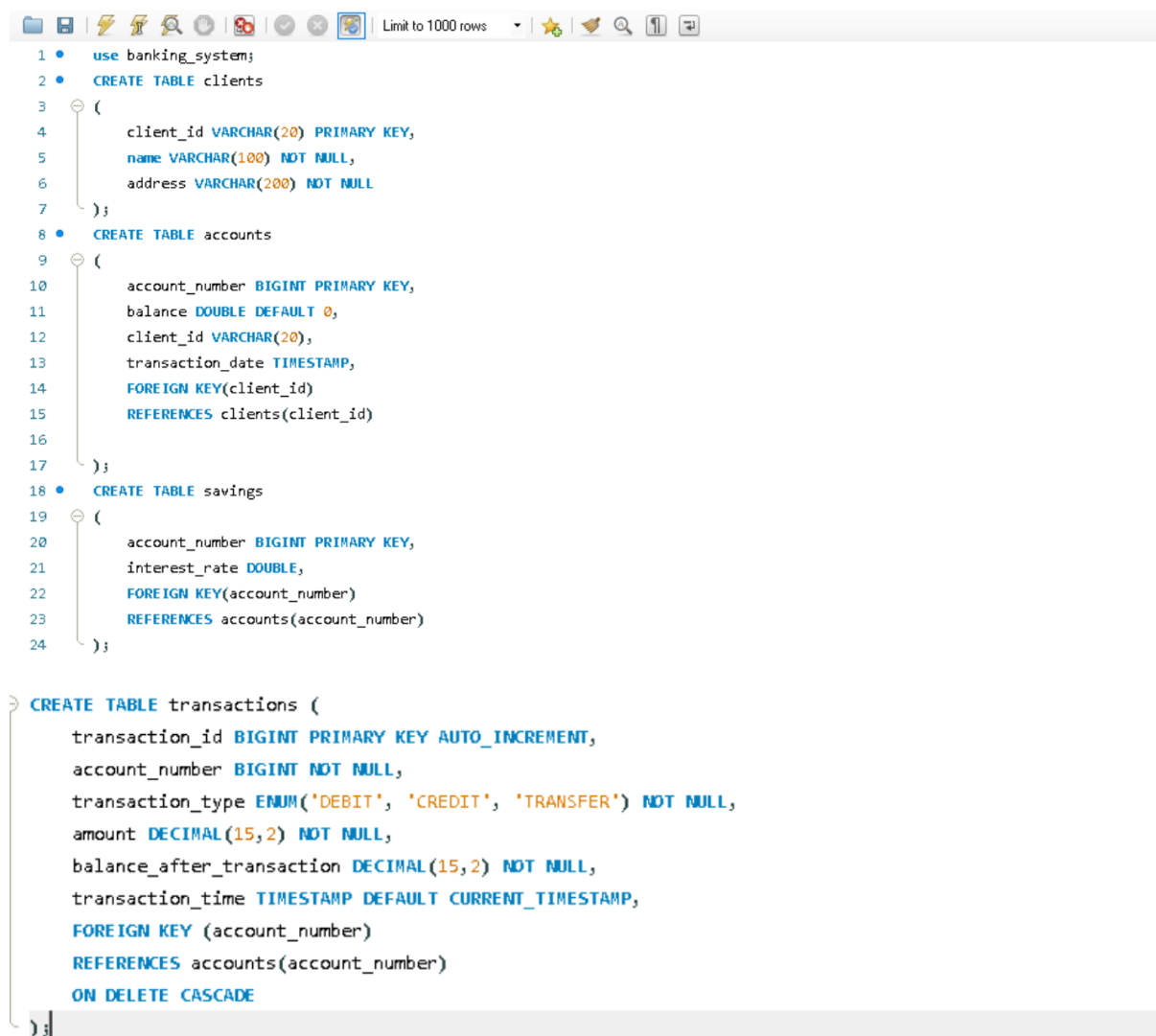
Stores all banking transactions.

Column Name	Data Type	Description
transaction_id	BIGINT	Unique Transaction ID
account_number	BIGINT	Account Number
transaction_type	ENUM	DEBIT, CREDIT, TRANSFER
amount	DECIMAL(15,2)	Transaction Amount
balance_after_transaction	DECIMAL(15,2)	Remaining Balance
transaction_time	TIMESTAMP	Transaction Time

Relationship:

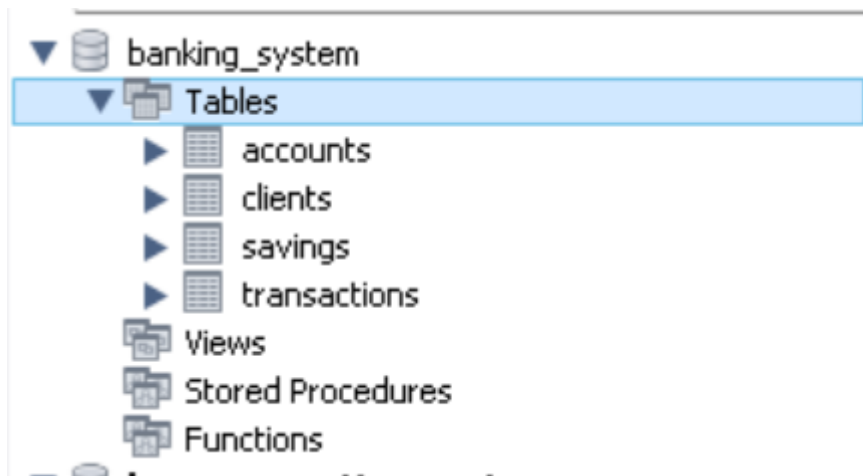
- account_number references accounts(account_number).

DATABASE SCRIPT SCREENSHOT:



```
1 • use banking_system;
2 • CREATE TABLE clients
3   (
4     client_id VARCHAR(20) PRIMARY KEY,
5     name VARCHAR(100) NOT NULL,
6     address VARCHAR(200) NOT NULL
7   );
8 • CREATE TABLE accounts
9   (
10    account_number BIGINT PRIMARY KEY,
11    balance DOUBLE DEFAULT 0,
12    client_id VARCHAR(20),
13    transaction_date TIMESTAMP,
14    FOREIGN KEY(client_id)
15    REFERENCES clients(client_id)
16  );
17
18 • CREATE TABLE savings
19   (
20    account_number BIGINT PRIMARY KEY,
21    interest_rate DOUBLE,
22    FOREIGN KEY(account_number)
23    REFERENCES accounts(account_number)
24  );
25
26 CREATE TABLE transactions (
27   transaction_id BIGINT PRIMARY KEY AUTO_INCREMENT,
28   account_number BIGINT NOT NULL,
29   transaction_type ENUM('DEBIT', 'CREDIT', 'TRANSFER') NOT NULL,
30   amount DECIMAL(15,2) NOT NULL,
31   balance_after_transaction DECIMAL(15,2) NOT NULL,
32   transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
33   FOREIGN KEY (account_number)
34   REFERENCES accounts(account_number)
35   ON DELETE CASCADE
36 );
```

DATABASE TABLE SCREENSHOT



CODE SNIPPETS

DATABASE CONNECTION

```
1  import java.sql.*;
2
3  public class DBConnection { 16 usages  @Devabisheik
4      public static Connection getConnection() throws SQLException { 16 usages  @Devabisheik
5          String url = "jdbc:mysql://localhost:3306/banking_system";
6          String user = "root";
7          String password = "Abi@2007";
8          Connection con = DriverManager.getConnection(url, user, password);
9          return con;
10     }
11 }
12
```

ACCOUNT CREATION

```
private static void createAccount() throws SQLException { 1 usage  @De
    scan.nextLine();
    System.out.print("Enter Your Name: ");
    String name = scan.nextLine();
    System.out.print("Enter Your Address: ");
    String address = scan.nextLine();
    String clientID = IDGenerator.generateID();
    long accNumber = IDGenerator.generateAccountNumber();
    System.out.print("Enter Initial Balance: ");
    double balance = scan.nextDouble();
    Client client = new Client(clientID, name, address);
    Account account = new Account(accNumber, client);
    account.balance = balance;
    ClientDAO.addClient(clientID, name, address);
    AccountsDAO.addAccount(clientID, accNumber, balance);
    System.out.println("\n==== ACCOUNT CREATED SUCCESSFULLY =====");
    System.out.println("Client ID      : " + clientID);
    System.out.println("Account Number : " + accNumber);
}
```

DEPOSIT MONEY:

```
public void deposit(long accNumber, double amount) { 1 usage  @Devabisheik
    try {
        double balance = AccountsDAO.getBalance(accNumber);
        BankServiceDAO.deposit_money(accNumber, amount);
        Transaction.addDetails(accNumber, amount, balanceAmount: balance+amount, type: "CREDIT");
        System.out.printf("Amount: %.2f was deposited to Account: %d\n", amount, accNumber);
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
```

WITHDRAW MONEY:

```
public void withdraw(long accNumber, double amount) { 1 usage  @Devabisheik
    try {
        double balance = AccountsDAO.getBalance(accNumber);
        BankServiceDAO.withdraw_money(accNumber, amount);
        Transaction.addDetails(accNumber, amount, balanceAmount: balance-amount, type: "DEBIT");
        System.out.printf("Amount: %.2f was withdrawn from Account: %d\n", amount, accNumber);
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
```

TRANSFER MONEY

```
public void transfer(long fromAcc, long toAcc, double amount) { 1 usage  @Devabisheik
    try {
        double fromBalance = AccountsDAO.getBalance(fromAcc);
        double toBalance = AccountsDAO.getBalance(toAcc);
        BankServiceDAO.transfer(fromAcc, toAcc, amount);
        Transaction.addDetails(fromAcc, amount, balanceAmount: fromBalance-amount, type: "TRANSFER");
        Transaction.addDetails(toAcc, amount, balanceAmount: toBalance+amount, type: "TRANSFER");
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
```

OUTPUT SCREENSHOTS

MAIN MENU

```
"C:\Users\acer\IdeaProjects\JAVA-ALFIDO-INTERN\BANKING SYSTEM\out\production\JAVA-DEVELOPER-INTERN;  
C:\Users\acer\Downloads\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" BankingSystem  
  
===== BANK MENU =====  
1) Create Account  
2) Show Client  
3) Add Savings  
4) Show Account  
5) Show All Client  
6) Show All Account  
7) Deposit Money  
8) Withdraw Money  
9) Transfer Money  
10) Show History  
11) Exit
```

ACCOUNT DETAILS

```
Enter Any Operation Number: 1  
Enter Your Name: ABISHEIK RAJAN  
Enter Your Address: 147 A.K.T school back side neelamangalam kallakurichi  
Enter Initial Balance: 1000  
Your details successfully inserted...  
Your account successfully inserted...  
  
===== ACCOUNT CREATED SUCCESSFULLY =====  
Client ID      : C7357  
Account Number : 9833816198
```

CLIENT DETAILS

```
Enter Any Operation Number: 2  
Enter Client ID: C7357  
  
===== CLIENT DETAILS =====  
ID: C7357  
Name: ABISHEIK RAJAN  
Address: 147 A.K.T school back side neelamangalam kallakurichi
```

ACCOUNT DETAILS

```
Enter Any Operation Number: 4
Enter Account Number: 9833816198

===== ACCOUNT DETAILS =====
Account Number: 9833816198
Balance: 1000.0
```

DISPLAY ALL CLIENTS

```
===== OUR BANK CLIENTS =====
=====
ID: C4649
Name: Abisheik
Address: a.k.t school back side
=====
ID: C4890
Name: rahul
Address: a.k.t school back side
=====
ID: C6188
Name: ABISHEIK R
Address: xyz street abc district
=====
ID: C7293
Name: ABISHEIK R
Address: 147 ABC street kallakurichi
=====
ID: C7357
Name: ABISHEIK RAJAN
Address: 147 A.K.T school back side neelamangalam kallakurichi
```

```

=====
ID: C7978
Name: Abdul Kalam
Address: 12/7 Mosque Street, Rameswaram, Tamil Nadu 623526
=====
ID: C9781
Name: Rahul
Address: akt school back side, Neelamangalam, Kallakurichi
=====
ID: C9814
Name: ABISHEIK R
Address: 147, A.K.T school back side vallar nagar Vivekananda street neelamangalam kallakurichi 606213
=====

```

DISPLAY ALL ACCOUNNTS

```

Enter Any Operation Number: 6
===== OUR BANK ACCOUNTS =====
=====
Account Number: 4311853401
Balance: 1500.0
=====
Account Number: 5744516291
Balance: 1400.0
=====
Account Number: 6949692438
Balance: 1000.0
=====
Account Number: 7095279409
Balance: 1200.0
=====
Account Number: 8818456271
Balance: 1000.0
=====
Account Number: 8868616093
Balance: 600.0
=====

```



```
=====
Account Number: 8868616093
Balance: 600.0
=====
Account Number: 9833816198
Balance: 1000.0
=====
Account Number: 9855095009
Balance: 1000.0
=====
```

MONEY TRANSFER:

```
Enter Any Operation Number: 9
Enter Sender Account Number: 9833816198
Enter Receiver Account Number: 9855095009
Enter Amount to Transfer: 100
transaction completed successfully...
Transaction completed...
Transaction completed...
```

DEPOSIT MONEY:

```
Enter Any Operation Number: 7
Enter Your Account Number: 9833816198
Enter Amount to Deposit: 200
Amount was credited to an account [9833816198]
Transaction completed...
Amount: 200.00 was deposited to Account: 9833816198
```

WITHDRAW MONEY:

```
Enter Any Operation Number: 8
Enter Your Account Number: 9833816198
Enter Amount to Withdraw: 300
Amount [300.0] was debited from your account to an account [9833816198]
Transaction completed...
Amount: 300.00 was withdrawn from Account: 9833816198
```

Transaction History

```
Enter Any Operation Number: 11
Enter Account Number: 9833816198
===== 11 =====
Transaction Type: TRANSFER
Account Number: 9833816198
Amount: 100.0
Balance After Transaction: 900.0
Time: 2026-06-10 13:59:54.0

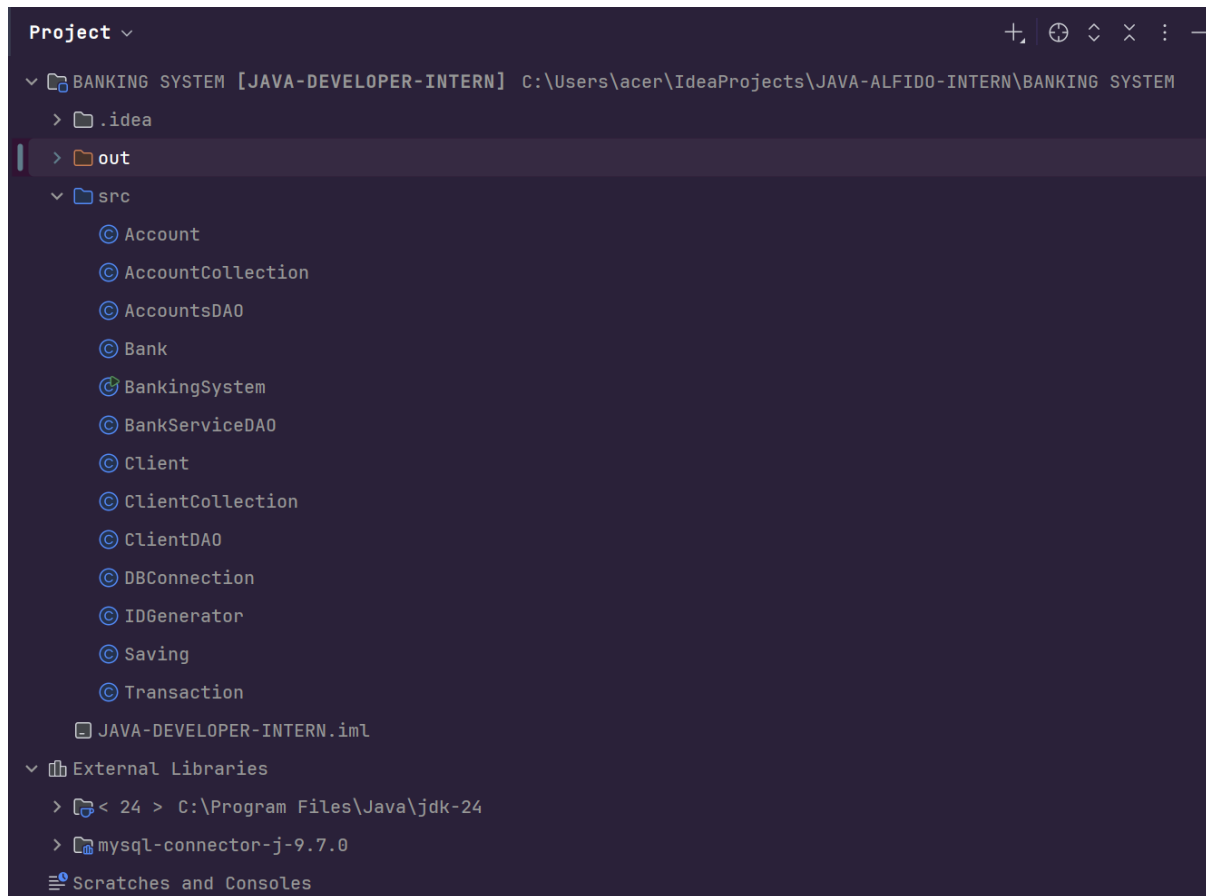
===== 13 =====
Transaction Type: CREDIT
Account Number: 9833816198
Amount: 200.0
Balance After Transaction: 1100.0
Time: 2026-06-10 14:40:47.0

===== 14 =====
Transaction Type: DEBIT
Account Number: 9833816198
Amount: 300.0
Balance After Transaction: 800.0
Time: 2026-06-10 14:41:02.0
```

RESULTS

The Banking Management System was successfully developed using Java, JDBC, and MySQL. The application was tested with different banking operations, and all functionalities worked as expected. Customer details and account information were stored and retrieved successfully from the database. The system was able to perform deposits, withdrawals, fund transfers, and transaction tracking accurately. The integration between Java and MySQL through JDBC was implemented successfully, ensuring reliable data management.

PROJECT FOLDER STRUCTURE



GITHUB REPOSITORY

Repository Link: [GitHub Link](#)

CONCLUSION

The Banking Management System provides an efficient way to manage customer accounts and banking transactions. Through this project, Java programming, Object-Oriented Programming concepts, JDBC connectivity, and MySQL database management were applied in a practical scenario. The project successfully demonstrates how software can be used to automate banking operations and maintain transaction records efficiently.